

Practica 2

Ejercicio 1

a_ Necesito N procesos seccion critica que represente el uso de un detector de metales por una persona, y utilizar un semaforo para asegurar que se use de a una persona a la vez detector

b_

```
// Necesito un semaforo para representar el uso de una persona a la
// vez del detector de metales
```

```
sem mutex = 1;
```

```
process SC(id = 0..N-1){
  // Espera el detector en caso que este en 0 mutex
  P(mutex)
  // Usa el detector
  V(mutex)
  // Libera el detector
}
```

c_

```
sem mutex = 3;
```

```
process SC(id = 0..N-1){
  // Espera el detector en caso que este en 0 mutex
  P(mutex)
  // Usa el detector
  V(mutex)
  // Libera el detector
}
```

d_

```

sem mutex = 3;
cant = 0

process SC(id = 0..N-1){
  cant = Random(1, 100)
  while (cant > 0) {
    cant = cant -1;
    // Espera el detector en caso que este en 0 mutex
    P(mutex)
    // Usa el detector
    V(mutex)
    // Libera el detector
  }
}

```

Ejercicio 2

a_ Precondicion: el N es multiplo de 4

```

historial_fallos_gravedad [N];
historial_fallos_id [N];
porcion = N/4

process chequeo(id = 0..3) {
  int limiteInferior = porcion * id;
  int limiteSuperior = limiteInferior + porcion - 1;
  for (i = limiteInferior ; i < limiteSuperior; i++){
    if (historial_fallos_gravedad[i] == 3) {
      write(historial_fallos_id [i])
    }
  }
}

```

b_

```

historial_fallos_gravedad [N];
historial_fallos_id [N];
contador_fallos [4] = ([4] 0)
sem mutex = 1;
porcion = N/4

process chequeo(id = 0..3) {
    int limiteInferior = porcion * id;
    int limiteSuperior = limiteInferior + porcion - 1;
    for (i = limiteInferior ; i < limiteSuperior; i++){
        P(mutex);
        contador_fallos[historial_fallos_gravedad[i]]++;
        V(mutex)
    }
}

```

c_

```

historial_fallos_gravedad [N];
historial_fallos_id [N];
contador_fallos [4] = ([4] 0)
porcion = N/4

process contador(id = 0..3) {
    for (i = 0; i < N; i++){
        if (historial_fallos_gravedad[i] == id)
            contador_fallos[id]++;
    }
}

```

Ejercicio 3

```

queue recursos[5];
sem mutex_recursos = 5
sem mutex = 1;

```

```
process usarCola(id=0..P-1) {
    recurso r;
    P(mutex_rekursos);
    P(mutex)
    pop(rekursos, r)
    V(mutex)
    // Usar recurso
    P(mutex)
    push(rekursos, r)
    V(mutex)
    V(mutex_rekursos);
}
```

Ejercicio 4

El problema de la solución presentada es que puede ocurrir que lleguen 6 procesos de prioridad alta por ejemplo, acaben el semaforo total, y queden dos procesos esperando que se desocupen los cupos de prioridad alta, cuando estos dos cupos de total, podrian usarse en ejecutar procesos de prioridad baja, en vez de quedarse esperando que se desocupen los de prioridad alta.

Una soucion podria hacer primero hacer los $P(\text{alta})$ y $P(\text{baja})$ primero antes de los $P(\text{total})$

Ejercicio 5

a_

```
sem paquetes = 0; sem cantVacios = N  
int contenedores[N] = ((N) 0)  
int vacios = 0  
int ocupados = 0
```

[o, -, -, -, -, -, -, -, -, -, -, -, -]

```
process preparador {
```

[O _ _ _ _ _ _ _ _ _ _]

```
process preparador {
```

```
Paquete paquete;  
while (true){  
    paquete = generarPaquete();  
    P(cantVacios);  
    contenedores[vacios] = paquete;  
    vacios = (vacios + 1) MOD N;  
    V(paquetes);  
}
```

```
process entregador {
    Paquete paquete;
    while (true) {
        P(paquetes)
        paquete = contenedores [ocupados];
        ocupados = (ocupados + 1) MOD N;
        //entregar paquete
        V(cantVacios)
        entregarPaquete(p)
    }
}
```

b)

```
sem paquetes = 0; sem canVacios = N; sem prep = 1;
int[] contenedores = ([N])
int pri_v = 0
int pri_o = 0

[x, -, -, -, -, -, -, -, -, -, -, -, -]

process preparador[id = 0 to P-1]{
    while (true){
        paquete = generarPaquete()
        p(cantVacios)
        p(prepare)
        contenedores[pri_v] = paquete;
        pri_v = pri_v + 1 MOD N;
```


d)

 $[x, _, _, _, _, _, _, _, _, _]$

```
process entregador [id = 0 to E-1]{
  while (true) {
    Paquete p;
```

```

P(paquetes)
p(ent);
  p = contenedores [pri_o];
  pri_o = pri_o + 1 MOD N;
v(ent);
//entregar paquete
v(cantVacios)
entregarPaquete(p)
}
}

```

Ejercicio 6

a_

```

sem mutex = 1;

process persona(id = 1 .. N) {
  P(mutex);
  Imprimir(documento);
  V(mutex);
}

```

b_

```

sem mutexN [N];
sem mutex = 1;
queue cola;
boolean libre = true;

process persona(id = 1 .. N) {
  int aux;
  P(mutex);
  if (libre) {
    libre = false;
    V(mutex);
  } else {

```



```

        push(cola, id);
        V(mutex);
        P(espera[id]);
    }
    Imprimir(documento);
    P(mutex)
    if (empty(cola))
        libre = true;
    else {
        pop(cola, aux);
        V(espera[aux]);
    }
    V(mutex)
}

```

c_

```

sem espera[1..N+1];
espera[1] = 1;

Process Persona[id:1..N]{
    while(true){
        P(espera[id])
        Imprimir(Documento)
        V(espera[id+1])
    }
}

```

d_

```

sem mutex = 1;           // protege la cola
sem espera[N] = {0};     // semáforo privado de cada persona
sem fin = 0;             // para que el coordinador sepa que terminó el turno
sem llegue = 0;
queue cola;

process persona(id = 1..N) {
    P(mutex);

```

```

push(cola, id);      // me anoto en la cola
V(mutex);
  V(llegue)          // avisa que llego alguien

P(espera[id]);      // espero a que el coordinador me autorice

Imprimir(documento); // uso exclusivo de la impresora

V(fin);             // aviso al coordinador que terminé
}

process coordinador {
  int aux;
  for (k = 1 .. N) { // repite hasta que todos impriman
    P(llegue)        // espera a que llegue alguna persona
    P(mutex);
    pop(cola, aux);  // saca al primero de la cola
    V(mutex);

    V(espera[aux]);  // habilita a esa persona

    P(fin);          // espera que termine de imprimir
  }
}

```

e_

```

em espera[P] = ([P] 0);
sem mutex = 1;
sem llena = 0;
colaLlegada c;
int impresoras[5] = {0,1,2,3,4};
sem cantImpres = 5;
int impersona[P] = ([P] -1);
sem mutexImpresoras = 1;

Process persona[i:0..P-1]{
  Documento documento;

```

```

P(mutex);
c.push(i);
V(mutex);
V(llena);
P(espera[i]);
Imprimir(documento, impersona[i]);
P(mutexImpresoras);
impresoras.push(impersona[i]);
V(mutexImpresoras);
V(impresoras);
}

```

```

Process coordinador{
    int aux;
    int impaux;
    for i = 0..P-1{
        P(llena);
        P(mutex);
        aux = c.pop();
        V(mutex);
        P(impresoras);
        P(mutexImpresoras);
        impaux = impresoras.pop();
        V(mutexImpresoras);
        impersona[aux] = impaux;
        V(espera[aux]);
    }
}

```